

Engenharia de Software II

André L. D. Rossi

Universidade Estadual Paulista “Júlio de Mesquita Filho” (UNESP)
Faculdade de Ciências (FC) / Departamento de Computação (DCo)
Bauru, SP - Brasil

Projeto baseado em Padrões

Projeto baseado em Padrões

Todos nós já nos deparamos com um problema de projeto e, silenciosamente, pensamos: *será que alguém já desenvolveu uma solução para esse problema?*

A resposta é quase sempre *sim!*

Algumas dificuldades:

- Encontrar a solução
- Garantir que, de fato, adapte-se ao problema em questão
- Entender as restrições que talvez limitem a maneira pela qual a solução é aplicada
- Traduzir a solução proposta para seu ambiente de projeto

Projeto baseado em Padrões

Mas e se a solução fosse codificada de alguma forma?

E se existisse uma maneira padronizada de descrever um problema (de tal forma que pudéssemos pesquisá-lo) e um método organizado para representar a solução para o problema?

Os problemas de software seriam codificados e descritos usando-se um modelo padronizado e seriam propostas soluções (com restrições) para eles.

Denominado **padrões de projeto**, esse método codificado para descrição de problemas e suas soluções permite que os profissionais da engenharia de software adquiram conhecimento de projeto para que ele seja reutilizado.

Projeto baseado em Padrões

O início da história dos **padrões** começa com um arquiteto, Christopher Alexander.

Alexander encontrou um conjunto de problemas recorrentes toda vez que um edifício era projetado. Ele caracterizou esses problemas recorrentes e suas soluções como **padrões**, descrevendo-os da seguinte maneira:

Cada padrão descreve um problema que ocorre repetidamente em nosso ambiente e então descreve o cerne de uma solução para aquele problema para podermos usá-la repetidamente um milhão de vezes sem jamais ter de fazer a mesma coisa duas vezes.

As ideias de Alexander foram traduzidas inicialmente para o mundo do software em livros como os de Gamma [1].

Hoje, existem dezenas de repositórios de padrões, e projetos baseados em padrões podem ser aplicados em diversos domínios de aplicação.

Padrões de Projeto

Um *padrão de projeto* pode ser caracterizado como “uma **regra de três partes** que expressa uma relação entre um **contexto**, um **problema** e uma **solução**”.

Para projeto de software, o *contexto* permite ao leitor compreender o ambiente em que o problema reside e qual solução poderia ser apropriada nesse ambiente.

Um conjunto de requisitos, incluindo limitações e restrições, atua como um *sistema de forças*¹ que influencia a maneira pela qual o problema pode ser interpretado em seu contexto e como a solução pode ser aplicada eficientemente.

¹Forças são as características do problema e os atributos de uma solução que restringem a maneira como o projeto pode ser desenvolvido.

A maioria dos problemas possui várias soluções, porém uma solução é eficaz somente se for apropriada no contexto do problema existente.

É o sistema de forças que faz com que um projetista escolha uma solução específica. O intuito é fornecer uma solução que melhor atenda ao sistema de forças, mesmo quando essas forças são contraditórias.

Por fim, toda solução tem consequências que poderiam ter um impacto sobre outros aspectos do software, e ela própria poderia fazer parte do sistema de forças para outros problemas a serem resolvidos no sistema mais amplo.

Um padrão de projeto evita que tenhamos de “reinventar a roda” ou, pior ainda, inventar uma “nova roda” que não será perfeitamente redonda; será muito pequena para o uso pretendido e muito estreita para o terreno onde irá rodar.

Os padrões de projeto, se usados de maneira eficiente, invariavelmente o tornarão um melhor projetista de software.

Tipos de Padrões

Tipos de Padrões

Uma das razões para os engenheiros de software se interessarem (e ficarem intrigados) por padrões de projeto é o fato de os seres humanos serem inerentemente bons no reconhecimento de padrões.

No mundo real, os padrões que reconhecemos são aprendidos ao longo de toda uma vida de experiências.

Reconhecemos instantaneamente e compreendemos inerentemente seus significados e como eles poderiam ser usados.

Alguns desses padrões nos dão uma melhor visão do fenômeno da recorrência.

Tipos de Padrões

No jargão dos padrões, um **padrão não generativo** apenas descreve um contexto e um problema, mas não fornece nenhuma solução explícita.

Quando são considerados padrões de projeto de software, faz-se um esforço para identificar e documentar padrões **generativos**.

Em um ambiente ideal, um conjunto de padrões de projeto generativos poderia ser usado para “gerar” uma aplicação ou sistema computacional cuja arquitetura permitisse que se **adaptasse à mudança**.

Os padrões de projeto abrangem um amplo espectro de abstração e aplicação.

Tipos de Padrões

Os **padrões de arquitetura** descrevem problemas de projeto de caráter amplo e diverso, resolvidos usando-se uma abordagem estrutural.

Os **padrões de dados** descrevem problemas orientados a dados recorrentes e as soluções de modelagem de dados que podem ser usadas para resolvê-los.

Tipos de Padrões

Os **padrões de componentes** (também conhecidos como padrões de projeto) tratam de problemas associados ao desenvolvimento de subsistemas e componentes, da maneira pela qual eles se comunicam entre si e de seu posicionamento em uma arquitetura maior.

Os **padrões de interfaces** descrevem problemas comuns de interfaces do usuário e suas soluções, com um sistema de forças que inclui as características específicas dos usuários.

Tipos de Padrões

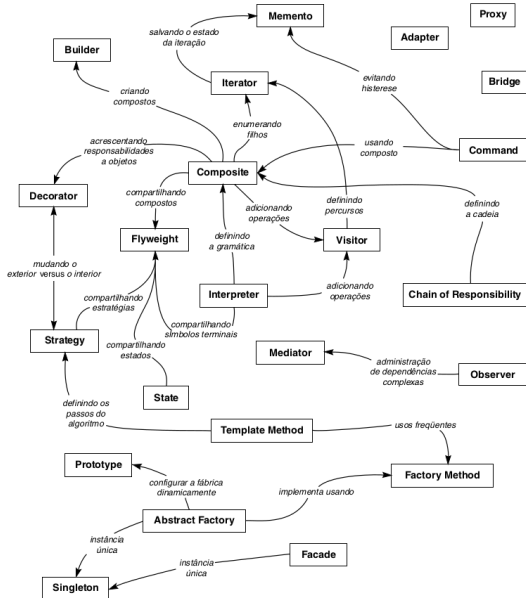
Os padrões para WebApp tratam de um conjunto de problemas encontrados ao se construir WebApps.

Em geral, incorporam muitas das demais categorias de padrões mencionados.

Em seu livro seminal sobre padrões de projeto, Gamma [1] e seus colegas descrevem três tipos de padrões particularmente relevantes para projetos orientados a objetos:

- padrões criacionais;
- padrões estruturais; e
- padrões comportamentais.

Relacionamento entre padrões de projeto



Padrões Criacionais

Os padrões criacionais se concentram na **criação, composição e representação** de objetos e dispõem de mecanismos que facilitam a instanciação de objetos.

Os padrões criacionais também impõem **restrições** sobre o **tipo** e **número** de objetos que podem ser criados em um sistema.

Exemplos de padrões criacionais:

- Singleton
- Método fábrica

Singleton

Suponha uma classe *Logger*, usada para registrar as operações realizadas em um sistema:

```
class Logger {  
    public Logger() { ... }  
  
    public void println(String msg) {  
        System.out.println("Log:" + msg);  
    }  
}  
  
public class LoggerMain {  
    public static void main(String[] args) {  
        Logger log1 = new Logger();  
        Logger log2 = new Logger();  
  
        log1.println("Primeira mensagem");  
        log2.println("Segunda mensagem");  
    }  
}
```

```
void f() {  
    Logger log = new Logger();  
    log.println("Executando f");  
    ...  
}  
  
void g() {  
    Logger log = new Logger();  
    log.println("Executando g");  
    ...  
}  
  
void h() {  
    Logger log = new Logger();  
    log.println("Executando h");  
    ...  
}
```

Singleton

Problema:

Cada método cria sua própria instância de `Logger`. Isso pode não ser adequado para acesso a algum recurso (banco de dados, arquivos, etc).

Solução:

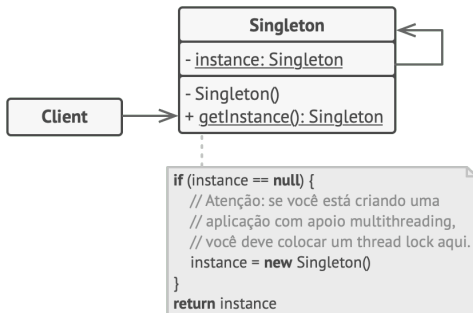


Figura 1: Padrão Criacional Singleton

(<https://refactoring.guru/pt-br/design-patterns/singleton>).

Singleton

```
class Logger {  
    // proíbe clientes de chamar new Logger()  
    private Logger() {}  
    // instancia unica  
    private static Logger instance;  
    public static Logger getInstance()  
    {  
        if (instance == null)  
            instance = new Logger();  
        return instance;  
    }  
    public void println(String msg)  
    {  
        // Poderia ser em arquivo  
        System.out.println(msg);  
    }  
}
```

```
void f() {  
    Logger log = Logger.getInstance();  
    log.println('Executando f');  
    ...  
}  
  
void g() {  
    Logger log = Logger.getInstance();  
    log.println('Executando g');  
    ...  
}  
  
void h() {  
    Logger log = Logger.getInstance();  
    log.println('Executando h');  
    ...  
}
```

Fábrica (estático)

Suponha um sistema distribuído baseado em TCP/IP. Nesse sistema, três funções f , g e h criam objetos do tipo *TCPChannel* para comunicação remota:

```
void f() {  
    TCPChannel c = new TCPChannel();  
    ...  
}
```

```
void g() {  
    TCPChannel c = new TCPChannel();  
    ...  
}
```

```
void h() {  
    TCPChannel c = new TCPChannel();  
    ...  
}
```

E se o sistema precisar usar UDP ao invés do TCP para comunicação?
Esse trecho de código não atende ao princípio Aberto/Fechado!

Fábrica (estático)

```
void f() {  
    TCPChannel c = new  
        TCPChannel();  
    ...  
}  
  
void g() {  
    TCPChannel c = new  
        TCPChannel();  
    ...  
}  
  
void h() {  
    TCPChannel c = new  
        TCPChannel();  
    ...  
}
```

```
class ChannelFactory {  
    // metodo fabrica estatico  
    public static Channel create() {  
        return new TCPChannel();  
    }  
}  
  
void f() {  
    Channel c=ChannelFactory.create();  
    ...  
}  
  
void g() {  
    Channel c=ChannelFactory.create();  
    ...  
}  
  
void h() {  
    Channel c=ChannelFactory.create();  
    ...  
}
```

Método Fábrica

Imagine agora que você poder sobrescrever o método fábrica em uma subclasse e mudar o produto que está sendo criado.

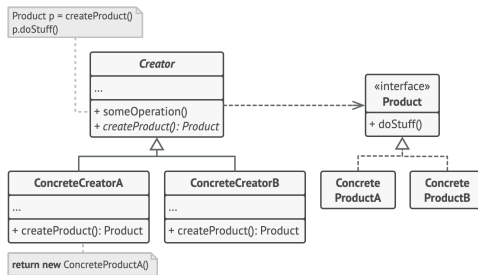


Figura 2: Uma fábrica é criada para cada produto.

Método Fábrica

Para o caso da “fábrica de canais de comunicação”:

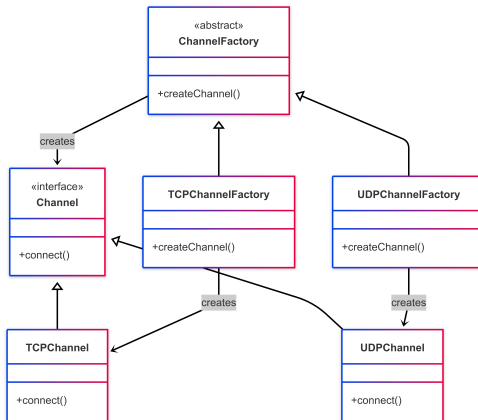


Figura 3: Padrão Criacional Método Fábrica para diferentes produtos (TCP e UDP).

Padrões Estruturais

Os padrões estruturais focalizam problemas e soluções associadas a como classes e objetos são **organizados e integrados** para construir uma estrutura maior.

Exemplos de padrões estruturais:

- Padrão adaptador
- Padrão de composição
- Padrão proxy

Suponha um sistema que tenha que controlar projetores multimídia. Para isso, o sistema instancia objetos de classes disponibilizadas pelos fabricantes de cada projeto:

```
class ProjetorSamsung {  
    public void turnOn() { ... }  
    ...  
}
```

```
class ProjetorLG {  
    public void enable(int timer) { ... }  
    ...  
}
```

Adaptador

No sistema de controle de projetores multimídia, queremos usar uma interface única para ligar os projetores, independentemente de marca.

O próximo código mostra essa interface e uma classe cliente do sistema:

```
interface Projetor {  
  
    void liga();  
  
}  
...  
class SistemaControleProjetores {  
  
    void init(Projetor projetor) {  
        projetor.liga(); // liga qualquer projetor  
    }  
  
}
```

Problema:

As classes de cada projeto foram implementadas pelos fabricantes e não temos acesso ao código dessas classes para fazer com que elas implementem a interface *Projetor*.

Solução:

O padrão Adaptador pode ser usado para converter a interface *Projetor* para as interfaces (métodos públicos) das classes implementadas pelos fabricantes.

Adaptador

Exemplo de classe adaptadora, de *ProjetoSamsung* para *Projeto*, é mostrado a seguir:

```
class AdaptadorProjetoSamsung implements Projeto {  
  
    private ProjetoSamsung projeto;  
  
    AdaptadorProjetoSamsung (ProjetoSamsung projeto) {  
        this.projeto = projeto;  
    }  
  
    public void liga() {  
        projeto.turnOn();  
    }  
}
```

Adaptador

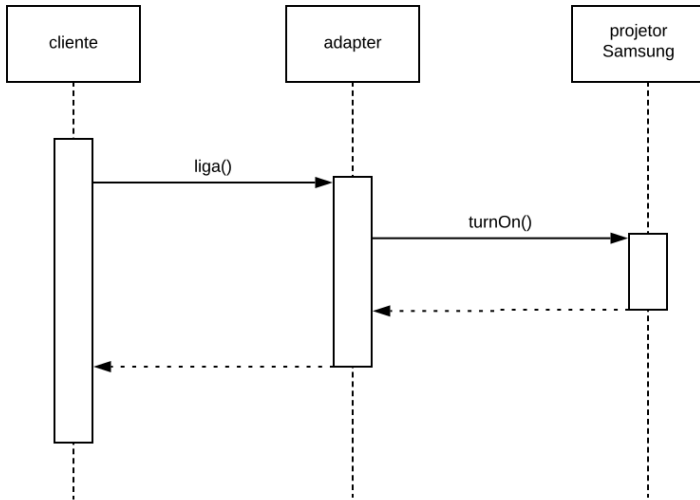


Figura 4: Diagrama de sequência UML mostrando as chamadas.

Padrões Comportamentais

Os padrões comportamentais tratam de problemas associados à **atribuição de responsabilidade** entre objetos e a maneira como a **comunicação** é realizada entre objetos.

Exemplos de padrões comportamentais:

- Observador;
- Strategy; e
- Padrão iterador.

Considere um sistema para controlar câmaras refrigeradas.

Nesse sistema, temos que manipular objetos de duas classes:

Temperatura : são objetos de “modelo” que armazenam as temperaturas monitoradas na estação meteorológica

Termometro : são objetos de “visão” que exibem as temperaturas sob monitoramento.

Termômetros devem exibir a temperatura atual que foi monitorada. Se a temperatura mudar, os termômetros devem ser atualizados.

Problema: Não é desejável acoplar Temperatura (classe de modelo) a Termometro (classe de interface). O motivo é simples: classes de interface mudam com frequência.

É possível ter diferentes visões (interfaces), tais como textual, Web e dispositivos móveis. Também pode-se oferecer outras interfaces de termômetros, tais como digital, analógico, etc.

Por fim, outras classes semelhantes podem existir, como UmidadeDoAr e Higrômetro.

Solução: O padrão **Observador** é a solução recomendada para esse contexto e problema.

Esse padrão define como implementar uma relação do tipo um-para-muitos entre objetos **sujeito** e **observadores**.

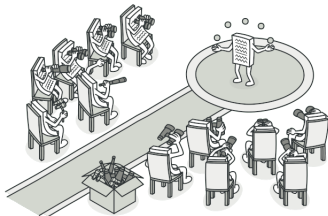


Figura 5: O objeto sujeito e os objetos observadores.

Observador

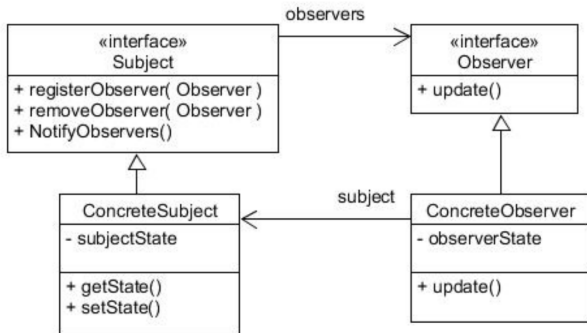


Figura 6: Diagrama UML para o padrão Observador. As interfaces *Observer* e *Subject* e suas respectivas classes concretas.

O padrão observador tem muitas similaridades com o padrão de arquitetura *publisher/subscriber*.

Exemplo de solução para temperatura (sujeito) e termômetro (observador).

Classe principal:

```
void main() {  
    Temperatura t = new Temperatura();  
    t.registerObserver(new TermometroCelsius());  
    t.registerObserver(new TermometroFahrenheit());  
    t.setTemp(100.0);  
}
```

Observador

```
class Temperatura extends Subject {  
    private double temp;  
  
    public double getTemp() {  
        return temp;  
    }  
  
    public void setTemp(double temp) {  
        this.temp = temp;  
        NotifyObservers();  
    }  
}  
  
class TermometroCelsius implements Observer {  
  
    public void update(Subject s){  
        double temp = ((Temperatura) s).getTemp();  
        System.out.println("Temperatura Celsius: " + temp);  
    }  
}
```